Pallas: The Planning Asset Administration Shell

Specification v1.0

Affiliation Redacted

Affiliation Redacted

Athor Redacted
Email Redacted

Athor Redacted
Email Redacted

Contents

Figures

1. Introduction

This document describes Pallas: Pallas: The Planning Asset Administration Shell. Pallas is an Asset Administration Shell (AAS) [3–5] model for representing semantic planning information. The model is open source (MIT license) and available on GitHub[1] The model is intended to store the same planning-relevant information that can be represented in Plato[2], while using AAS submodels and submodel elements as the underlying representation. Its purpose is to enable the structured provision of planning knowledge in Industry 4.0 (I4.0) environments and to support the automatic generation of planning domain descriptions for Artificial Intelligence (AI) planning.

The model is designed to represent, among other things, planning objects, types, predicates, actions, costs, functions, initial and goal states, and optimization metrics in a machine-interpretable manner. In contrast to ontology-based representations, the model uses standard AAS concepts such as Submodel Element Collections and Submodel Element Lists to structure this information. This allows planning knowledge to be integrated into digital twins and exchanged within AAS-based environments. In addition, Pallas is not restricted to a single AAS. Instead, the semantic planning information can be distributed across multiple AASs, which allows planning knowledge to be maintained close to the assets to which it belongs.

This document is structured as follows. Section 2 outlines the motivation for developing the AAS model. Section 3 gives a quick overview of the AAS concepts used in Pallas. Section 4 describes the conceptual structure of the model and its individual elements. Finally, Section 5 refers to the tooling and transformation support that already exist for this model.

2. Motivation

In AI planning, the planning domain and planning problem are commonly modeled in the Planning Domain Definition Language (PDDL) [1, 2]. While PDDL is well suited for formal planning, it is less suited as a primary knowledge representation for complex and evolving industrial environments. Creating and maintaining planning descriptions manually requires significant expert effort and makes the resulting models difficult to adapt when the underlying environment changes.

At the same time, Asset Administration Shells are becoming increasingly important as standardized digital representations of assets in Industry 4.0 environments. They are used to organize structured information about physical and non-physical assets and provide a common basis for interoperability between systems. As a result, AASs are a natural candidate for storing planning-relevant information in a form that is both semantically structured and integrated into industrial data spaces.

The AAS model described in this document is intended to bridge these two concerns. It provides a structured representation of planning knowledge inside AAS submodels such that this knowledge

[1] https://github.com/user140613/Pallas
[2] https://github.com/user140613/Plato

can later be transformed automatically into planning domain descriptions. The model is designed to capture the information required for AI planning, including object types, planning objects, predicates, actions with preconditions and effects, arithmetic cost expressions, functions, and optimization metrics. In this way, the model allows planning knowledge to be maintained in an AAS-based ecosystem while preserving its usability for automated planning. Since Pallas supports the distribution of planning information across multiple AASs, it also enables decentralized modeling scenarios in which different parts of the planning knowledge are maintained by different assets or systems.

3. AAS Foundations

The model is based on the standard structure of the Asset Administration Shell. An AAS consists of an `AssetAdministrationShell`, its `AssetInformation`, and one or more `Submodels` [5]. The planning information described here is stored in a dedicated submodel.

Within that submodel, the model uses the following AAS element types:

- **Property** to store scalar values such as names, operators, and numeric values
- **Reference Element** to reference other elements of the planning model, potentially across multiple AASs
- **Submodel Element Collection (SMC)** to represent complex structured elements
- **Submodel Element List (SML)** to represent ordered or unordered lists of homogeneous elements

This choice of AAS structures allows the model to represent both atomic planning information and nested composite expressions.

4. Specification

This section describes the conceptual structure of the Pallas. The model is realized through one or more planning-information submodels that can be distributed across multiple AASs. The submodels contain several top-level Submodel Element Lists (SMLs) and a property. These lists organize the planning knowledge into distinct categories. The contained planning elements are modeled primarily using Submodel Element Collections (SMCs), Properties, and Reference Elements. Figure 4.1 provides an overview of the conceptual structure of Pallas.

At the top level, the model consists of ten SMLs and one property:

- `Name (property specifing the name of the domain)`
- `Types`

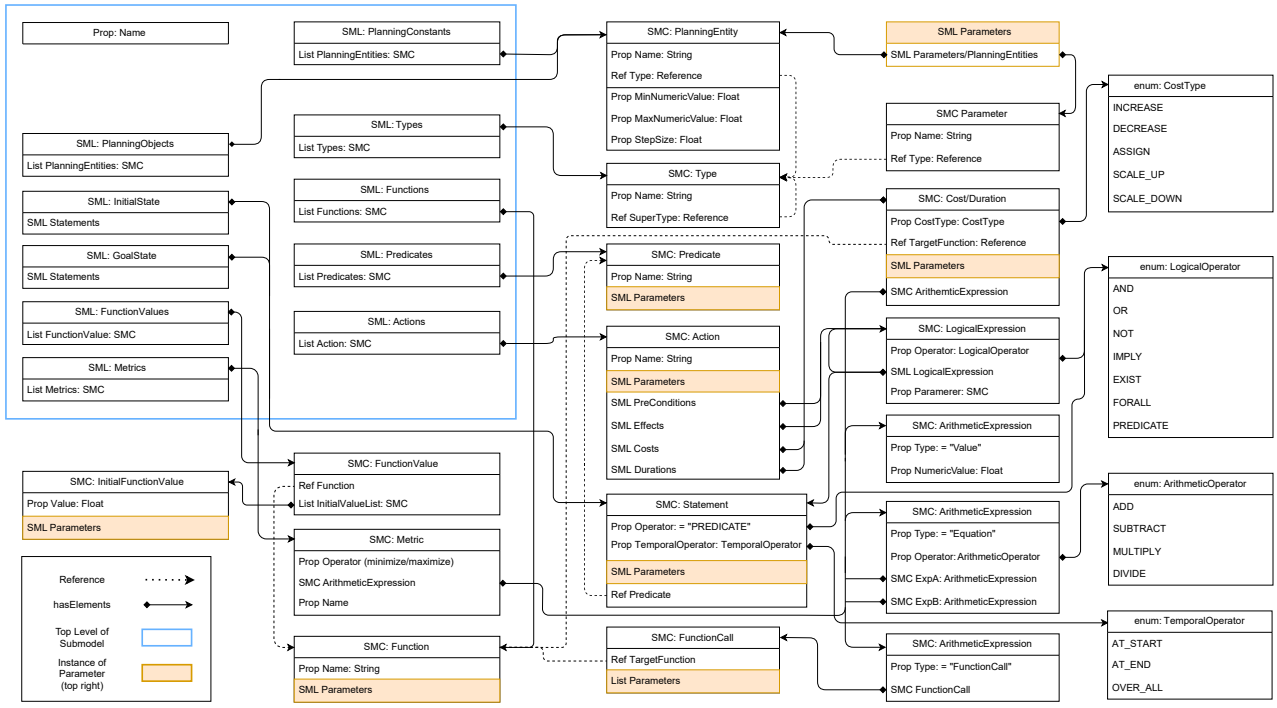

Figure 4.1: Conceptual model for planning information

- `PlanningConstants`
- `PlanningObjects`
- `Predicates`
- `Functions`
- `FunctionValues`
- `Actions`
- `InitialState`
- `GoalState`
- `Metrics`

These top-level lists store all planning-relevant information required for classical and temporal planning scenarios supported by the model. Conceptually, Pallas consists of these ten categories of information. In a concrete deployment, however, these categories do not need to be stored in a single AAS. Instead, the semantic information can be distributed across multiple AASs. For example, the SML `PlanningObjects` can appear in multiple AASs so that planning objects can be specified in a decentralized manner by different assets. The same principle can also be applied to other categories of planning information if required. During parsing, the information distributed across these AASs is combined into one planning representation.

4.1 Object Types

Types are stored in the SML `Types`. Each element of this list is an SMC representing one type. A type consists of a unique name and a reference to its supertype.

This bottom-up representation of the type hierarchy allows every type, except the root type, to reference exactly one supertype. As a result, hierarchical type structures can be represented explicitly. Subtype relationships are not stored directly but can be derived from the supertype references.

The type system is used throughout the model. Planning objects, planning constants, predicate parameters, action parameters, and function parameters all reference these types. This reduces modeling effort because predicates and actions can be defined on more general types and reused for all compatible sub-types.

4.2 Planning Constants and Planning Objects

Planning entities are divided into two categories: planning constants and planning objects. Both are structurally represented in the same way, but they serve different roles in planning.

Planning objects represent entities that are relevant only to a specific planning problem. Examples include individual workpieces or other instance-specific objects. Planning constants represent entities that are always relevant in the domain, such as stationary resources or structural entities. In Pallas, both `PlanningObjects` and `PlanningConstants` can be distributed across multiple AASs. This makes it possible, for example, for several assets to provide their own planning objects independently.

Constants and objects are stored in the `PlanningConstants` and `PlanningObjects` SMLs that each consist of `PlanningEntity` SMCs. Each planning entity is represented with at least the following elements:

- a Property `Name` of type string
- a Reference Element `Type` referencing an element in `Types`

If the planning entity represents a numeric constant, three additional properties can be present:

- `MinNumericValue`
- `MaxNumericValue`
- `StepSize`

These optional numeric properties allow automated discretization of numeric constants to allow for the generation of planning domains that work on non-numeric planners.

4.3 Parameters

Several concepts in the model require parameters. This includes predicates, statements, actions, functions, and function calls. To ensure a uniform structure, all of them use the same parameter representation.

A parameter is represented as an `Parameter` SMC containing:

- a Property `Name`
- a Reference Element `Type`

Multiple parameters are stored in an ordered SML. The order is important because a predicate, action, or function can have multiple parameters of the same type. The ordered list ensures that parameter positions remain unambiguous. The SML Parameters used in many constructs of Pallas can contain parameter definitions (i.e., name and type) or planning entities. For example, a definition of a predicate used a Parameters SML only filled with definitions, but the effect of an action might also reference planning constants when using a Parameters SML through a statement. In the latter case, the list could also contain parameter definitions because the effect might also reference actual parameters of the action.

4.4 Predicates

Predicates are used to represent state information in the planning domain. They are Boolean functions that describe properties of entities or relations between them.

All predicates are stored in the SML `Predicates`. Each predicate is represented as an `Predicate` SMC with:

- a Property `Name`
- an ordered SML `Parameters`

A predicate definition specifies only the structure of the predicate, that is, its name and the names and types of its parameters. It does not yet bind the predicate to concrete planning entities. Concrete uses of predicates are represented through statements, which are described in Section 4.8.

4.5 Functions and Function Values

Functions are used to represent numeric mappings. They are particularly relevant for costs, durations, and metrics. All functions are stored in the SML `Functions`. Each function is represented as an `Function` SMC containing:

- a Property `Name`
- an ordered SML `Parameters`

The function definition specifies the general form of the function. The actual numeric mappings are stored separately in SMCs of the `FunctionValue` type, which are collected in the top-level SML `FunctionValues`. Each FunctionValues entry references one function, and provides a list of initial function values for it. The `InitialFunctionValue` SMC is used to specify a single initial value for a given parameter combination. This separation between function definitions and function values allows the model to represent general function signatures together with concrete numeric assignments for specific parameter combinations.

4.6 Actions

Actions represent the transition functions of the planning domain. They define how the domain state can change by specifying parameters, preconditions, effects, costs, and, for temporal domains, durations. All actions are stored in the SML `Actions`. Each action is represented as an SMC containing:

- a Property `Name`
- an ordered SML `Parameters`
- an SML `PreConditions`
- an SML `Effects`
- an SML `Costs`
- an SML `Durations`

The preconditions and effects are modeled as logical expressions. The costs and durations are modeled as arithmetic expressions contained in a `Cost/Duration` SMC. This SMC specifies which function should be increased/decreased, etc., with which parameters. Specifically, this SMC contains

- a Property `CostType` with one of the following values: `INCREASE`, `DECREASE`,`ASSING`, `SCALE_UP`, or `SCALE_DOWN`,
- a reference element `TargetFunction`
- an ordered SML `Parameters`
- an SMC `ArithmeticExpression`

It should be noted that the actions might also have effects that change numeric functions but would not be considered costs. These effects are still modeled using the `Cost` SMC.

4.7 Logical Expressions

Preconditions and effects are represented as tree-structured logical expressions using the `LogicalExpression` SMC. Each logical expression is modeled with a property that stores the logical operator and an SML that stores nested logical expressions. The following logical operators are supported:

- `AND`
- `OR`
- `NOT`
- `IMPLY`
- `EXIST`
- `FORALL`
- `WHEN`

For `AND` and `OR`, the list of nested logical expressions can contain any number of elements. For `NOT`, exactly one nested expression is allowed. For `IMPLY`, exactly two nested expressions are required.

The operators `EXIST` and `FORALL` each require one nested logical expression together with an additional parameter specifying the relevant type. The `WHEN` expression is used together with `FORALL` to model a condition under which the universal quantification applies.

The leaf nodes of logical expression trees are always represented through the `Statement` SMC using the `PREDICATE` operator. Figure 4.2 summarizes all supported logical expression types.

4.8 Statements

Statements are used to state a boolean fact about the world. They can be used to model the initial and goal state and to help with preconditions and effects. A statement is represented as an `Statment` SMC containing:

- a Property `Operator` with the value `PREDICATE`
- a optional Property `TemporalOperator` with one of the following values: `AT_START`, `AT_END`, ot `OVER_ALL`
- an ordered SML `Parameters`
- a Reference Element `Predicate`

Through this structure, the same predicate definition can be reused in multiple contexts with different concrete arguments.

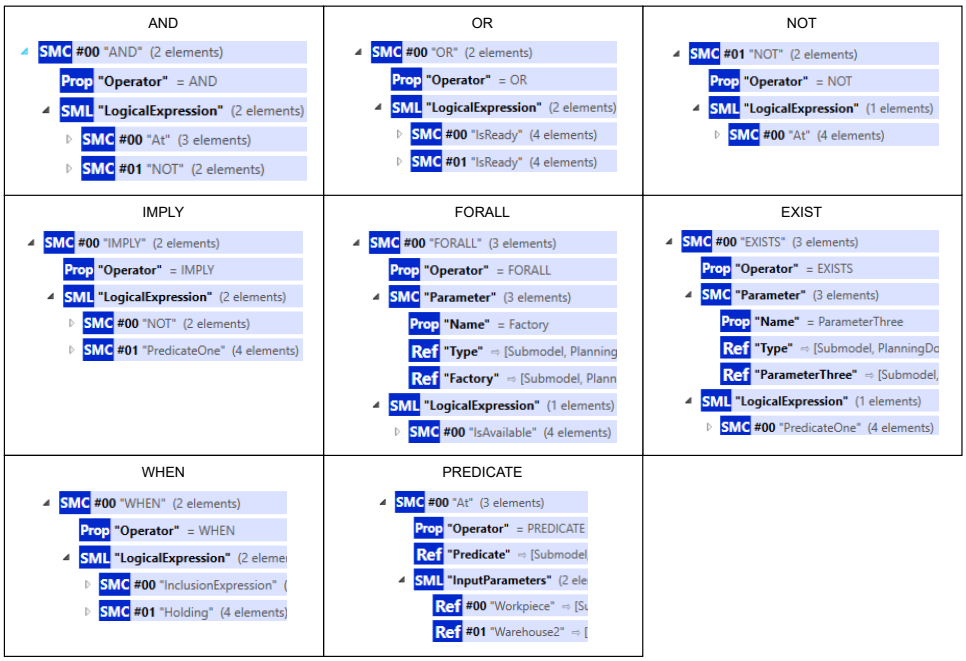

Figure 4.2: All supported logical expression types

4.9 Arithmetic Expressions

Arithmetic expressions are used to model action costs, durations, and optimization metrics. Like logical expressions, they are represented in nested structures that act as binary expression trees. Each node is represented by a `ArithmeticExpression` SMC. Each arithmetic expression contains a Property `Type`, which determines its structure. The following expression types are supported:

- `Value`
- `FunctionCall`
- `Equation`

An expression of type `Value` represents a static numeric value and contains one additional Property `NumericValue`.

An expression of type `FunctionCall` represents a value obtained from a function. It contains a `FunctionCall` SMC with a reference to a function and an ordered list of input parameters.

An expression of type `Equation` combines two arithmetic expressions using an arithmetic operator. The supported arithmetic operators are:

- `ADD`
- `SUBTRACT`
- `MULTIPLY`
- `DIVIDE`

This representation allows both simple costs and more complex cost equations to be modeled in a uniform way. Figure 4.3 summarizes the three supported arithmetic expression types.

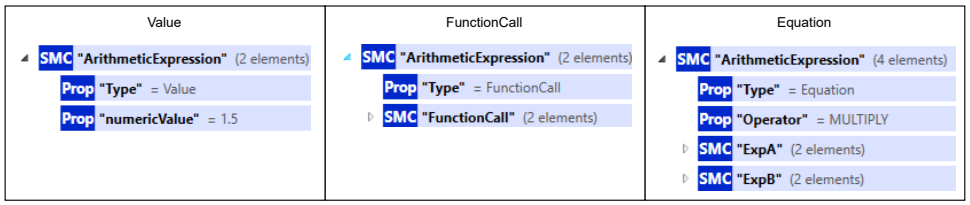

Figure 4.3: All three different arithmetic expressions

4.10 Initial and Goal State

The initial and goal state of a planning problem are represented as sets of statements. The SML `InitialState` contains the statements that are true at the beginning of the planning problem. The SML `GoalState` contains the statements that must hold when the planning problem is solved.

The model follows the usual closed-world assumption used in planning. This means that entities not listed as true in the initial state are considered false.

4.11 Metric

Pallas supports optimization metrics in addition to ordinary goal conditions. Metrics are stored in the top level `Metrics` SML that contains a list of `Metric` SMCs. A metric SMC contains:

- a `Operator` Property specifying whether the expression is to be "minimized" or "maximized"
- an arithmetic expression SMC
- an optional `Name` property

This allows the model to represent optimization criteria such as minimizing resource consumption, or maximizing a utility value.

5. Available Tooling

The only official tool that currently supports Pallas is the Knowledge Transformation Framework (KTF). This framework provides, among other things, a set of tools for transforming knowledge represented in Pallas into various formats, such as PDDL planning domains. It also supports the distributed modeling approach of Pallas by allowing semantic planning information from multiple AASs to be combined during transformation. For more information, refer to the documentation of KTF provided in its GitHub repository[3].

[3] https://github.com/user140613/KTF

References

[1] Ghallab, M., Knoblock, C., Wilkins, D., Barrett, A., Christianson, D., Friedman, M., Kwok, C., Golden, K., Penberthy, S., Smith, D., Sun, Y., Weld, D.: Pddl - the planning domain definition language (08 1998), https://courses.cs.washington.edu/courses/cse473/06sp/pddl.pdf

[2] Haslum, P., Lipovetzky, N., Magazzeni, D., Muise, C.: An Introduction to the Planning Domain Definition Language. Synthesis Lectures on Artificial Intelligence and Machine Learning, Morgan & Claypool Publishers, California, USA (2019). https://doi.org/10.2200/S00900ED2V01Y201902AIM042

[3] Industrial Digital Twin Association, Plattform Industrie 4.0: Asset administration shell - reading guide (11 2022), https://industrialdigitaltwin.org/wp-content/uploads/2022/12/2022-12-07_IDTA_AAS-Reading-Guide.pdf, version 2022-12-07

[4] Plattform Industrie 4.0: Details of the Asset Administration Shell - Part 2: Application programming interfaces (API) (November 2021), https://www.plattform-i40.de/IP/Redaktion/EN/Downloads/Publikation/Details_of_the_Asset_Administration_Shell_Part2_V1.html, accessed: [Insert Date]

[5] Plattform Industrie 4.0: Details of the Asset Administration Shell - Part 1: The exchange of information between partners in the value chain of Industrie 4.0 (May 2022), https://www.plattform-i40.de/IP/Redaktion/EN/Downloads/Publikation/Details_of_the_Asset_Administration_Shell_Part1_V3.html, accessed: [Insert Date]